

Open-FDD Documentation

Generated by scripts/build_docs_pdf.py

June 7, 2026

- 1 Home
- 2 Open-FDD
 - 2.1 What you get
 - 2.2 Two paths
 - 2.2.1 Try it with Docker (recommended)
 - 2.2.2 Develop locally
 - 2.3 Distribution
 - 2.4 License
- 3 Run with Docker images
- 4 Run with Docker images
 - 4.1 Images
 - 4.2 1. Install and validate Docker
 - 4.3 2. Bootstrap the site (one script)
 - 4.3.1 Manual start (optional)
 - 4.4 Long-term operation
 - 4.4.1 Survive power cycles
 - 4.4.2 Start / stop / restart
 - 4.4.3 LAN access
 - 4.5 Next steps
- 5 First login and health check
- 6 First login and health check
 - 6.1 Public health
 - 6.2 Login
 - 6.3 Smoke checklist
 - 6.4 Stack detail (authenticated)
 - 6.5 Troubleshooting
- 7 Quick Start
- 8 Quick Start
- 9 Updating the stack
- 10 Updating the stack
 - 10.1 Before you start
 - 10.2 Fetch helper scripts (no git clone)
 - 10.3 One-command upgrade (recommended)
 - 10.4 Step by step
 - 10.4.1 1. Backup site state
 - 10.4.2 2. Verify images (optional)
 - 10.4.3 3. Pull and recreate

- 10.4.4 4. Verify
- 10.5 Dashboard UI updates
- 10.6 Rollback
- 10.7 Safe cleanup
- 10.8 Next steps
- 11 Local development
- 12 Local development
 - 12.1 Clone and auth
 - 12.2 Start stack
 - 12.3 BACnet bench overlay (optional)
 - 12.4 Docs preview
- 13 Build Docker images
- 14 Build Docker images
 - 14.1 Local build
 - 14.2 Publish to GHCR (maintainers)
- 15 Developer Guide
- 16 Developer Guide
- 17 Tests and CI
- 18 Tests and CI
 - 18.1 Local tests
 - 18.2 CI (GitHub Actions)
 - 18.3 Docs build (same as CI)
- 19 Contributing
- 20 Contributing
 - 20.1 Expectations
 - 20.2 Pull request checklist
 - 20.3 Repository layout (short)
 - 20.4 Security issues
 - 20.5 Agent-assisted development
- 21 Health checks (developer)
- 22 Health checks (developer)
 - 22.1 Clone and local stack
 - 22.2 Stack health script
 - 22.3 Compose overlays
 - 22.4 BACnet poll pipeline
 - 22.5 pytest (bridge security / BACnet)
 - 22.6 Pentest production stack (LAN ZAP)
 - 22.7 Post-deploy insurance (inventory hosts)
- 23 System overview
- 24 System overview
 - 24.1 Docker edge stack (production)
 - 24.2 Components
 - 24.3 Deployment
 - 24.4 Optional PyPI-only path
- 25 Arrow recipes
- 26 Arrow recipes (default)
 - 26.1 Simple threshold
 - 26.2 Supply air temp high
 - 26.3 Fan commanded but no airflow
 - 26.4 Economizer not opening
 - 26.5 Heating and cooling simultaneously
 - 26.6 Sensor out of range

27	Data flow
28	Data flow
28.1	Telemetry ingest
28.2	Rule evaluation
28.3	Model sync
28.4	Retention
29	Python recipes (Arrow)
30	Python recipes (Arrow)
31	Containers
32	Containers
32.1	GHCR images
32.2	Stack diagram
32.3	Responsibilities
32.4	Network and ports (typical edge)
32.5	Long-lived deployment
32.6	Persistent data
32.7	Optional host services
33	Architecture
34	Architecture
35	Rule Cookbook
36	Rule Cookbook
37	Windowing & debugging
38	Windowing & debugging
38.1	Rolling windows (Arrow)
38.2	Rule Lab console
39	Dashboard overview
40	Dashboard overview
40.1	Primary areas
40.2	Live updates
40.3	Roles
41	Auth and roles
42	Auth and roles
42.1	Production (LAN / edge)
42.2	Localhost dev
43	Rule Lab
44	Rule Lab
45	Model workflow
46	Model workflow
46.1	Brick model
46.2	Typical integrator flow
47	Operator Bridge
48	Operator Bridge
49	Network setup
50	Network setup
50.1	Bind address
50.2	Discovery range
50.3	Verify
51	Discover and read
52	Discover and read
52.1	Operator workflow
53	Polling

- 54 Polling
 - 54.1 Commission poll loop (default)
 - 54.2 Bridge ingest worker
 - 54.3 Health
- 55 Write safety
- 56 Write safety
 - 56.1 Defaults
 - 56.2 Enable writes (commission role)
 - 56.3 Operator rules
- 57 BACnet
- 58 BACnet
- 59 Convention
- 60 Fault code convention
 - 60.1 Format
 - 60.2 Categories
 - 60.3 Severity
 - 60.4 Linking rules
 - 60.5 Cookbook mapping
- 61 AHU faults
- 62 AHU faults
- 63 VAV faults
- 64 VAV faults
- 65 RTU faults
- 66 RTU faults
- 67 Sensor and data quality
- 68 Sensor and data quality
 - 68.1 Communication quality
 - 68.2 Alarm console quality
- 69 Fault Codes
- 70 Fault Codes
- 71 Live site update (SSH)
- 72 Live site update (SSH)
 - 72.1 Concepts (plain language)
 - 72.2 Before you start
 - 72.3 1. Backup workspace/
 - 72.4 2. Verify new images exist on GHCR
 - 72.5 3. Update the React UI bundle (required for dashboard changes)
 - 72.6 4. Update docker-compose.yml and recreate containers
 - 72.7 5. On-VM smoke checks
 - 72.7.1 Arrow / FDD rules (Open-FDD 3.0+)
 - 72.8 6. Control-machine insurance check
 - 72.9 7. Browser validation
 - 72.10 Rollback
 - 72.11 Alternative: Ansible image-only upgrade
- 73 LAN hardening
- 74 LAN hardening
 - 74.1 Deployment modes
 - 74.2 Network exposure
 - 74.3 Rule Lab caution
- 75 Secrets and auth

- 76 Secrets and auth
 - 76.1 Files (gitignored)
 - 76.2 Required variables (production)
 - 76.3 GHCR pulls
 - 76.4 Backup
- 77 BACnet write allowlists
- 78 BACnet write allowlists
- 79 Operations
- 80 Operations
- 81 Security
- 82 Security
- 83 API routes
- 84 API routes
 - 84.1 Health & audit
 - 84.2 Auth
 - 84.3 Rules & FDD
 - 84.4 Rule Lab playground
 - 84.5 BRICK / data model
 - 84.6 BACnet
 - 84.7 Faults (check-engine)
 - 84.8 Timeseries & sites
 - 84.9 Building & host
 - 84.10 Local agent (Ollama)
 - 84.11 Modbus
 - 84.12 PyPI package (separate)
 - 84.13 Errors
- 85 Python package
- 86 Python package (open-fdd)
 - 86.1 Modules
 - 86.2 When to use package-only
 - 86.3 Versioning
 - 86.4 Offline Arrow example
- 87 Configuration reference
- 88 Configuration reference
 - 88.1 Bridge / auth
 - 88.2 BACnet
 - 88.3 Docker edge
- 89 Glossary
- 90 Glossary
- 91 Appendix
- 92 Appendix
- 93 Edge Docker deploy
- 94 Edge Docker deploy
- 95 Arrow-native runtime
- 96 Arrow-native runtime
 - 96.1 Authoring contract
 - 96.2 Package layout

Generated June 7, 2026

1 Home

2 Open-FDD

Open-FDD is an open-source platform for **fault detection**, **BACnet integration**, and **operator analytics** at the building edge. It combines a React dashboard, FastAPI Operator Bridge, Python Rule Lab, local feather historian, and optional Brick/RDF modeling.

Who it is for

Audience	Start here
IT / controls engineer trying the stack	Quick Start — Docker
Developer contributing or customizing	Developer Guide
Integrator wiring APIs	Appendix — API routes
Analyst authoring FDD rules	Rule Cookbook · Fault Codes

2.1 What you get

Component	Purpose
Operator Bridge	Web API + dashboard (trends, faults, Rule Lab, model tools)
BACnet tools	Discover, read, poll, optional supervised writes
Rule Lab	Arrow-native <code>apply_faults_arrow(table, cfg)</code> rules on feather historian data
Local historian	Feather-based telemetry store on the edge host
Brick / RDF model	Equipment and point semantics for bindings and analytics
Docker images	Published on GHCR — pull and run without building from source
Python package	

Component	Purpose
	<code>pip install open-fdd</code> for offline Arrow rule lint/test (<code>arrow_runtime</code>)

2.2 Two paths

2.2.1 Try it with Docker (recommended)

Pull published images from `ghcr.io/bbartling/`, configure auth and BACnet env files, start the stack, open the dashboard.

→ [Quick Start](#)

2.2.2 Develop locally

Clone the repo, create `workspace/auth.env.local`, build images, run tests, submit a PR.

→ [Developer Guide](#)

2.3 Distribution

Channel	Use when
GHCR images <code>ghcr.io/bbartling/openfdd-*</code>	Production or trial edge deploy
This repository	Custom builds, BACnet commissioning, Rule Lab development
PyPI <code>open-fdd</code>	Arrow runtime + playground (no full UI); optional [ml] for graph experiments

2.4 License

MIT — see repository LICENSE.

3 Run with Docker images

4 Run with Docker images

Deploy Open-FDD on a Linux edge host using **three published GHCR images**. No git clone required.

4.1 Images

GHCR image	Service	Role
ghcr.io/bbartling/openfdd-bridge	bridge	Operator API, dashboard, historian ingest
ghcr.io/bbartling/openfdd-commission	commission	BACnet discover/ read/write and poll loop
ghcr.io/bbartling/openfdd-mcp-rag	mcp-rag	Doc-search sidecar

Compose defaults to **latest**. Pin a dated tag with export OPENFDD_IMAGE_TAG=2026.06.07-edge when needed.

4.2 1. Install and validate Docker

```
sudo systemctl enable --now docker
sudo usermod -aG docker "$USER"
newgrp docker    # or log out/in

docker --version
docker compose version
docker ps
docker run --rm hello-world
```

Expect: active / enabled for docker, docker in groups, hello-world succeeds.

4.3 2. Bootstrap the site (one script)

Downloads docker-compose.yml, creates workspace/, generates auth.env.local, and sets BACNET_BIND from the host LAN NIC (e.g. ens192 → 10.200.200.185/24:47808):

```
curl -fsSL -o /tmp/openfdd_edge_bootstrap.sh \
https://github.com/bbartling/open-fdd/raw/refs/heads/master/
scripts/openfdd_edge_bootstrap.sh
bash /tmp/openfdd_edge_bootstrap.sh
```

Bootstrap + pull + start in one go:

```
bash /tmp/openfdd_edge_bootstrap.sh --start
```

Auth — bootstrap writes ~/open-fdd/workspace/auth.env.local (gitignored). This file holds the API signing secret and one username/password per role. Use it for your first dashboard login:

```
cat ~/open-fdd/workspace/auth.env.local
```

Variable	Role
OFDD_AUTH_SECRET	Signs session tokens (do not share)
OFDD_OPERATOR_*	Read-only operator
OFDD_INTEGRATOR_*	Default dashboard login — commissioning + BACnet
OFDD_AGENT_*	Agent / automation

Example shape (passwords are **random on first bootstrap only** — existing file is kept on re-runs):

```
OFDD_AUTH_SECRET=<random>
OFDD_OPERATOR_USER=operator
OFDD_OPERATOR_PASSWORD=<random>
OFDD_INTEGRATOR_USER=integrator
OFDD_INTEGRATOR_PASSWORD=<random>
OFDD_AGENT_USER=agent
OFDD_AGENT_PASSWORD=<random>
```

Sign in as **integrator** with the password from that file → First login and health check.

Action	Touches auth.env.local?
First bash ... bootstrap.sh --start	Creates file with random passwords (if missing)

Action	Touches auth.env.local?
bash ... bootstrap.sh --start again	Keeps your file — does not overwrite
bash ... bootstrap.sh --restart	Keeps your file — only restarts bridge to reload it
nano auth.env.local + --restart	Uses your edited passwords
--force-auth	Regenerates random passwords (opt-in only)

The **bridge** container reads auth.env.local at startup only. After you change passwords (manual edit or --force-auth), recreate bridge (env files are not reloaded by restart alone):

```
cd ~/open-fdd && docker compose up -d --force-recreate bridge
```

Regenerate passwords and reload (stack already running):

```
bash /tmp/openfdd_edge_bootstrap.sh --force-auth --restart --show-secrets
```

Manual edit then reload:

```
nano ~/open-fdd/workspace/auth.env.local
bash /tmp/openfdd_edge_bootstrap.sh --restart
```

Options:

Flag	Purpose
--start	docker compose pull && up -d after layout; curls http://127.0.0.1:8765/health
--image-tag TAG	default latest (falls back to 2026.06.07-edge)
--repo-ref BRANCH	branch for compose.edge.yml if not on master yet
--force-auth	regenerate auth.env.local (restarts bridge if stack is up)
--restart	recreate bridge to reload auth.env.local; curls /health after
--show-secrets	print passwords at end (lab only)

From a repo checkout: ./scripts/openfdd_edge_bootstrap.sh --start

The script prints **BACnet NIC**, **BACNET_BIND**, and file paths — **validate** commission.env before polling OT devices:

```
nano ~/open-fdd/workspace/bacnet/commissioning/commission.env
```

If you used `--start`, the stack is already up. Next step → First login and health check.

4.3.1 Manual start (optional)

Use this only if you ran bootstrap **without** `--start` and want to bring the stack up yourself:

```
cd ~/open-fdd
docker compose pull
docker compose up -d
docker compose ps
curl -sf http://127.0.0.1:8765/health && echo
```

Expected: **bridge, commission, mcp-rag** — all up. Then → First login and health check.

4.4 Long-term operation

4.4.1 Survive power cycles

All services use restart: unless-stopped. After reboot:

```
cd ~/open-fdd && docker compose ps
curl -sf http://127.0.0.1:8765/health
```

4.4.2 Start / stop / restart

```
cd ~/open-fdd
docker compose up -d           # idempotent
docker compose stop
docker compose restart commission
docker compose logs -f --tail 100 commission
```

Never run `docker compose down -v` or delete workspace/ on a live site.

4.4.3 LAN access

Put **Caddy** (or `nginx`) on port 80 → 127.0.0.1:8765, or expose 8765 through the firewall.

4.5 Next steps

→ [First login and health check](#)

→ [Updating the stack](#) — `./scripts/openfdd_site_update.sh`

5 First login and health check

6 First login and health check

Quick checks after `docker compose up -d` on the edge host. Services use `restart: unless-stopped` — after a reboot, run `docker compose ps` once Docker is up (see [Run with Docker](#)).

6.1 Public health

```
curl -s http://127.0.0.1:8765/health
```

Through Caddy on port 80:

```
curl -s http://127.0.0.1/health
```

Expect `"ok": true` and `"auth_required": true` when auth is configured.

6.2 Login

1. Open the dashboard (`http://<host-lan-ip>/` or `:8765`).
2. Sign in with credentials from `workspace/auth.env.local`.

```
curl -s -X POST http://127.0.0.1:8765/api/auth/login \
-H 'Content-Type: application/json' \
-d '{"username": "integrator", "password": "<from-auth-env>"}
```

6.3 Smoke checklist

Check	Pass
<code>docker compose ps</code>	bridge, commission, mcp-rag Up
<code>GET /health</code>	200, ok: true
Login	Token returned
BACnet poll	

Check	Pass
Historian	samples.csv growing (tail -1 workspace/bacnet/polls/samples.csv) Files under workspace/data/feather_store/

6.4 Stack detail (authenticated)

Integrator token required:

```
TOKEN=$(curl -sf -X POST http://127.0.0.1:8765/api/auth/login \
-H 'Content-Type: application/json' \
-d '{"username":"integrator","password":"..."}' | jq -r .token)
curl -sf http://127.0.0.1:8765/health/stack -H "Authorization:
Bearer $TOKEN" | jq .
```

Look for bacnet_poll green/yellow in the services list.

6.5 Troubleshooting

Symptom	Fix
401 everywhere	Configure workspace/auth.env.local
Empty BACnet discover	Fix BACNET_BIND in commission.env — <u>network setup</u>
No poll rows	Commission container running? points.csv has enabled rows?
LAN browser timeout	Open firewall port 80 or 8765

Advanced probes (model graph, Rule Lab, compose profiles):
Developer health check.

7 Quick Start

8 Quick Start

Run Open-FDD on a Linux edge host using **three GHCR Docker images**. No git clone on the host.

Step	Page
1	Run with Docker images — bootstrap script + docker compose up
2	First login and health check
3	Updating the stack — backup + pull

One-liner bootstrap (edge host):

```
curl -fsSL -o /tmp/openfdd_edge_bootstrap.sh \  
https://github.com/bbartling/open-fdd/raw/refs/heads/master/  
scripts/openfdd_edge_bootstrap.sh  
bash /tmp/openfdd_edge_bootstrap.sh --start
```

Prerequisites: Linux, Docker enabled on boot, BACnet LAN if polling OT equipment.

| **BACnet writes are disabled by default.** See [Write safety](#).

Stack: bridge + commission (BACnet + poll) + mcp-rag. Details: [Containers](#).

9 Updating the stack

10 Updating the stack

Upgrade GHCR images on the edge host: **backup workspace/, pull new tags, recreate containers**. No git pull on the host.

10.1 Before you start

1. SSH to the edge host (cd ~/open-fdd).
2. Pick a tag from [GitHub Packages](#) — default is **latest**.

3. Short maintenance window (containers restart briefly;
restart: unless-stopped brings them back after reboot).

10.2 Fetch helper scripts (no git clone)

If you bootstrapped with `openfdd_edge_bootstrap.sh`, scripts are already under `~/open-fdd/scripts/`. Otherwise:

```
mkdir -p ~/open-fdd/scripts
curl -fsSL -o ~/open-fdd/scripts/openfdd_site_backup.sh \
    https://github.com/bbartling/open-fdd/raw/refs/heads/master/
scripts/openfdd_site_backup.sh
curl -fsSL -o ~/open-fdd/scripts/openfdd_site_update.sh \
    https://github.com/bbartling/open-fdd/raw/refs/heads/master/
scripts/openfdd_site_update.sh
chmod +x ~/open-fdd/scripts/openfdd_site_*.sh
```

10.3 One-command upgrade (recommended)

```
cd ~/open-fdd
./scripts/openfdd_site_backup.sh
./scripts/openfdd_site_update.sh
```

Pulls `:latest` by default, verifies all three images exist on GHCR, recreates containers, and hits `/health`.

Pin a dated release:

```
export NEW_TAG=2026.06.07-edge
./scripts/openfdd_site_backup.sh
./scripts/openfdd_site_update.sh
```

10.4 Step by step

10.4.1 1. Backup site state

```
cd ~/open-fdd
./scripts/openfdd_site_backup.sh
```

Archives workspace/ (feather, BACnet CSVs, model, auth) under `~/openfdd-backups/<timestamp>/`.

Custom backup location:

```
BACKUP_ROOT=~/openfdd-backups/manual ./scripts/
openfdd_site_backup.sh
```

10.4.2 2. Verify images (optional)

```
export NEW_TAG="${NEW_TAG:-latest}"

docker manifest inspect ghcr.io/bbartling/openfdd-bridge:${NEW_TAG} >/dev/null &&
docker manifest inspect ghcr.io/bbartling/openfdd-commission:${NEW_TAG} >/dev/null &&
docker manifest inspect ghcr.io/bbartling/openfdd-mcp-rag:${NEW_TAG} >/dev/null &&
echo "All images exist for ${NEW_TAG}"
```

10.4.3 3. Pull and recreate

```
cd ~/open-fdd
export NEW_TAG="${NEW_TAG:-latest}" # optional pin
./scripts/openfdd_site_update.sh
```

Manual equivalent (same as the script):

```
cd ~/open-fdd
docker compose pull
docker compose up -d --force-recreate
docker compose ps
curl -sf http://127.0.0.1:8765/health && echo
```

10.4.4 4. Verify

```
docker compose ps
curl -sf http://127.0.0.1:8765/health | jq .
tail -1 workspace/bacnet/polls/samples.csv
ls -lah workspace/data/feather_store/
```

→ Health check

10.5 Dashboard UI updates

If the release changed the React UI, copy a new workspace/api/static/app/ build onto the host before or after the image pull. Image pull alone may not change the browser bundle hash.

10.6 Rollback

```
cd ~/open-fdd
export NEW_TAG=2026.06.06-edge # previous known-good tag
cp ~/openfdd-backups/<timestamp>/docker-compose.yml.snapshot
```

```
docker-compose.yml
./scripts/openfdd_site_update.sh
```

workspace/ is untouched by image-only upgrades.

10.7 Safe cleanup

```
docker image prune -f
```

Never run `docker compose down -v`, `docker volume prune`, or delete workspace/ on a live site.

10.8 Next steps

- [Run with Docker images](#) — bootstrap a new host
- [Health check](#)

11 Local development

12 Local development

12.1 Clone and auth

```
git clone https://github.com/bbartling/open-fdd.git && cd open-fdd
cp workspace/auth.env.example workspace/auth.env.local
# Edit integrator/operator passwords and OFDD_AUTH_SECRET (32+
chars)
```

12.2 Start stack

```
./scripts/build_and_test.sh           # dashboard build + pytest
./scripts/docker_build.sh
./scripts/openfdd_stack.sh up
./scripts/stack_health_check.sh
```

URL	Service
http://127.0.0.1:8765	Bridge (direct)
http://127.0.0.1:8765/docs	OpenAPI

12.3 BACnet bench overlay (optional)

```
docker compose -f docker/compose.dev.yml -f docker/
compose.bench.yml up -d
# commission.env — set BACNET_BIND to your OT interface
```

12.4 Docs preview

```
cd docs && bundle install && bundle exec jekyll serve
# http://127.0.0.1:4000/open-fdd/
```

13 Build Docker images

14 Build Docker images

14.1 Local build

```
./scripts/docker_build.sh
# Optional tar for air-gap:
./scripts/docker_build.sh --save
```

Images: openfdd-bridge, openfdd-commission, openfdd-mcp-rag.

14.2 Publish to GHCR (maintainers)

GitHub Actions workflow **Publish Docker addons** — manual dispatch with a version tag.

Maintainer checklist:

1. Green CI on master
2. Run workflow with tag e.g. 2026.06.06-edge
3. Verify packages at ghcr.io/bbartling/openfdd-*
4. Deploy to a demo edge with `OPENFDD_IMAGE_TAG=<tag>`

Details live in `.github/workflows/` and `scripts/docker_build.sh` — not duplicated here.

15 Developer Guide

16 Developer Guide

Clone, build, test, and contribute to Open-FDD.

Topic	Page
Local setup	Local development
Docker builds	Build Docker images
Health checks	Health checks (developer)
Tests & CI	Tests and CI
PRs	Contributing

AI-assisted contributors: see repository `AGENTS.md` for workspace layout, skill routing, and agent policy (one paragraph in public docs; details stay internal).

17 Tests and CI

18 Tests and CI

18.1 Local tests

```
./scripts/build_and_test.sh # UI + workspace bridge
pytest
python3 -m pytest open_fdd/tests -q # PyPI package tests
cd workspace/dashboard && npm test # dashboard unit tests (if
configured)
```

18.2 CI (GitHub Actions)

Workflow	Trigger	Checks
ci.yml	PR / push	pytest, bridge security audit, dashboard build, Jekyll docs
docs-pages.yml	push master	

Workflow	Trigger	Checks
publish-open-fdd.yml	tag open-fdd-v*	GitHub Pages site
publish-docker-addons.yml	manual	PyPI wheel
		GHCR images

18.3 Docs build (same as CI)

```
cd docs && bundle exec jekyll build -d /tmp/open-fdd-site
```

Fix any template or link errors before opening a PR that touches docs/.

19 Contributing

20 Contributing

20.1 Expectations

- Open an issue or discuss large changes before big refactors.
- Keep PRs focused; include test or docs updates when behavior changes.
- Do not commit secrets (auth.env.local, Ansible secrets/*.env.local, commissioning CSVs with real site data).
- Follow existing code style in workspace/api/ and workspace/dashboard/.

20.2 Pull request checklist

☐

Tests pass locally (./scripts/build_and_test.sh or scoped pytest)

☐

Docs updated if user-facing behavior changed

☐

docs/ Jekyll build succeeds

☐

No credentials in diff

20.3 Repository layout (short)

Path	Contents
workspace/api/	Operator Bridge (FastAPI)
workspace/dashboard/	React SPA
open_fdd/	PyPI package (arrow_runtime, playground)
docker/	Compose files and image contexts
infra/ansible/	Edge deploy playbooks
docs/	This site (Jekyll + Just the Docs)

20.4 Security issues

Do not file public issues for vulnerabilities. Contact the maintainer or use GitHub private security advisories.

20.5 Agent-assisted development

Cursor/Claude contributors: read AGENTS.md and skills/ for automation boundaries. Public docs intentionally avoid agent playbook detail.

21 Health checks (developer)

22 Health checks (developer)

Extended validation for engineers with a **full git checkout** — CI, compose overlays, and pre-release gates.

IT operators on edge hosts should use Quick Start — health check instead.

22.1 Clone and local stack

```
git clone https://github.com/bbartling/open-fdd.git && cd open-fdd
cp workspace/auth.env.example workspace/auth.env.local #
loopback dev only
./scripts/docker_build.sh # or: ./
scripts/openfdd_stack.sh prod
./scripts/openfdd_stack.sh up
```

GHCR production pull (no local build):

```
OPENFDD_IMAGE_TAG=2026.06.07-edge ./scripts/openfdd_stack.sh prod
```

22.2 Stack health script

```
./scripts/stack_health_check.sh
OPENFDD_BASE_URL=http://192.168.204.18:8765 ./scripts/
stack_health_check.sh
./scripts/stack_health_check.sh --require-ollama
```

Probes: containers up, /health, /api/model/health, sites, tree, SPARQL graph (with integrator auth when configured).

22.3 Compose overlays

Overlay	Use
docker/compose.dev.yml	Local dev bind-mount
docker/compose.bench.yml	Host-network BACnet lab
docker/compose.prod.yml	GHCR images instead of local build
docker/compose.edge.yml	IT edge template (copy to ~/open-fdd/ docker-compose.yml)

```
docker compose -f docker/compose.dev.yml -f docker/
compose.bench.yml ps
docker compose -f docker/compose.dev.yml config --images
```

22.4 BACnet poll pipeline

```
tail -1 workspace/bacnet/polls/samples.csv
cat workspace/data/bacnet_ingest_state.json
docker compose logs --since 10m commission | grep -i poll
docker compose logs --since 10m bridge | grep -i ingest
```

22.5 pytest (bridge security / BACnet)

```
PYTHONPATH=workspace/api pytest tests/workspace_bridge/
test_security.py -q
```

22.6 Pentest production stack (LAN ZAP)

```
./scripts/pentest_production_stack.sh start
./scripts/pentest_production_stack.sh verify
```

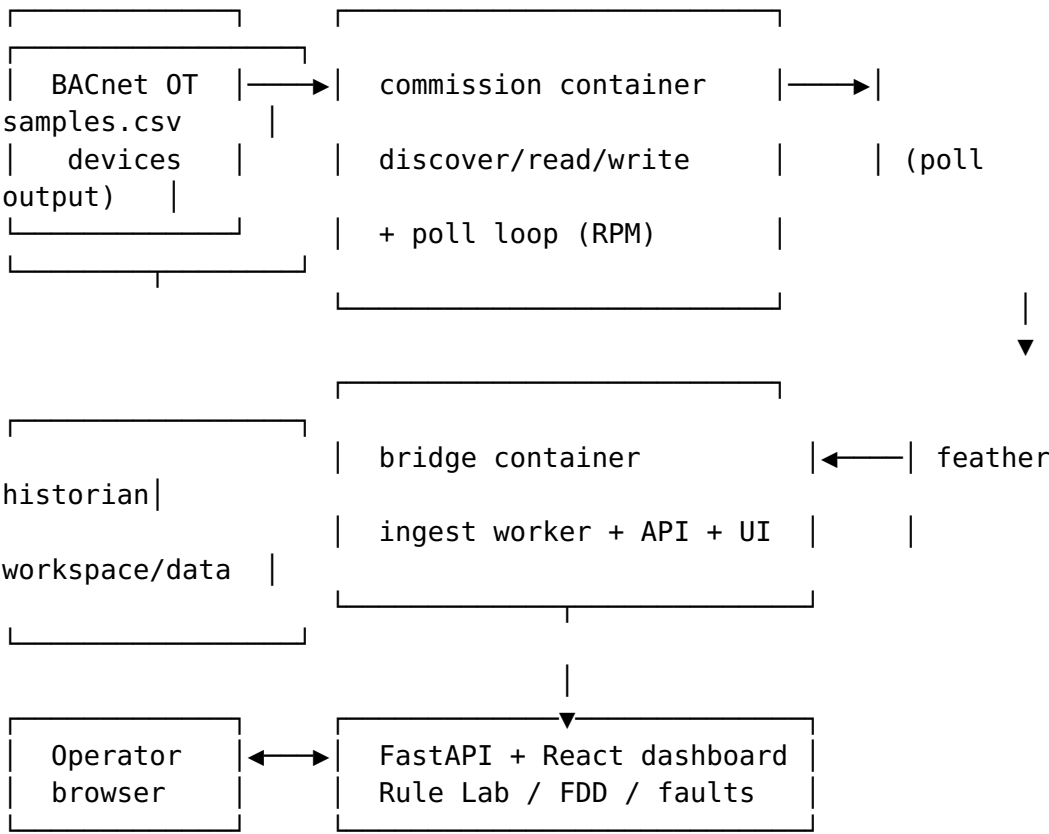
See workspace/deploy/PENTEST.md.

22.7 Post-deploy insurance (inventory hosts)

For automated checks against named inventory hosts, see infra/ansible/scripts/post_deploy_check.sh (maintainer tooling — not required for IT docker-pull sites).

23 System overview

24 System overview



▲
Caddy :80 (host, optional)

24.1 Docker edge stack (production)

Three GHCR images — see [Containers](#):

Container	BACnet OT	Operator LAN
bridge	No (HTTP only)	Yes, via Caddy :80 or loopback :8765
commission	Yes (host network, :47808)	No (internal :8767)
mcp-rag	No	No (internal :8090)

BACnet **polling** is not a separate container — it runs inside **commission**. The bridge only **ingests** poll CSV into feather.

24.2 Components

Layer	Responsibility
Operator Bridge	Auth, REST/WebSocket API, compiled dashboard, Rule Lab, historian ingest
BACnet commission	Who-Is, read/write (gated), scheduled poll loop → samples.csv
Historian	Feather files per point; local-first retention
FDD runner	workspace/data/rules_py/*.py on timer or POST /api/rules/batch
Model	Brick TTL + model.json bindings (fdd_input, equipment tree)
MCP RAG	Optional doc retrieval for agent tools
Caddy	Host reverse proxy :80 → bridge (typical edge)
Ollama	Optional host LLM for building insight (not required for FDD)

24.3 Deployment

IT operators: [Quick Start — Docker](#) (docker compose pull, restart: unless-stopped, Docker enabled on boot).

24.4 Optional PyPI-only path

`pip install open-fdd ships Arrow runtime + playground lint helpers` **without** the Operator Bridge UI. Use for offline rule tests or CI. See [Appendix — Python package](#).

25 Arrow recipes

26 Arrow recipes (default)

Open-FDD 3.0 Rule Lab rules use `apply_faults_arrow(table, cfg, context)` and `pyarrow.compute`.

26.1 Simple threshold

```
import pyarrow.compute as pc

def apply_faults_arrow(table, cfg, context=None):
    return pc.greater(table["zone_temp"], cfg["max_zone_temp"])
```

26.2 Supply air temp high

```
import pyarrow.compute as pc

def apply_faults_arrow(table, cfg, context=None):
    col = str(cfg.get("column", "SAT"))
    return pc.greater(table[col], float(cfg["high"]))
```

Config: {"column": "SAT", "high": 75}

26.3 Fan commanded but no airflow

```
import pyarrow.compute as pc

def apply_faults_arrow(table, cfg, context=None):
    fan_on = pc.greater(table["fan_cmd"],
    cfg.get("fan_on_threshold", 0.5))
    low_airflow = pc.less(table["airflow_cfm"],
    cfg["min_airflow_cfm"])
    return pc.and_(fan_on, low_airflow)
```

26.4 Economizer not opening

```
import pyarrow.compute as pc

def apply_faults_arrow(table, cfg, context=None):
    oat_good = pc.less(table["outside_air_temp"],
cfg["economizer_oat_limit"])
    cooling = pc.greater(table["cooling_cmd"],
cfg.get("cooling_threshold", 0.5))
    damper_low = pc.less(table["oa_damper_cmd"],
cfg["min_oa_damper_cmd"])
    return pc.and_(pc.and_(oat_good, cooling), damper_low)
```

26.5 Heating and cooling simultaneously

```
import pyarrow.compute as pc

def apply_faults_arrow(table, cfg, context=None):
    heating = pc.greater(table["heat_cmd"],
cfg.get("heat_threshold", 0.1))
    cooling = pc.greater(table["cool_cmd"],
cfg.get("cool_threshold", 0.1))
    return pc.and_(heating, cooling)
```

26.6 Sensor out of range

```
import pyarrow.compute as pc

def apply_faults_arrow(table, cfg, context=None):
    too_low = pc.less(table["sensor_value"], cfg["min_value"])
    too_high = pc.greater(table["sensor_value"], cfg["max_value"])
    return pc.or_(too_low, too_high)
```

More templates ship in `open_fdd.playground.arrow_templates` and via GET `/api/playground/arrow-templates`.

27 Data flow

28 Data flow

28.1 Telemetry ingest

1. **Commission poll loop** (inside the commission container) reads enabled BACnet points from `points.csv` on a schedule (host network, BACnet :47808).
2. RPM results append to `workspace/bacnet/polls/samples.csv` (long format).
3. **Bridge ingest worker** watches `samples.csv` and loads new rows into the **feather store** (`workspace/data/feather_store/`).
4. Point registry (`points.csv`, discovered inventory) ties BACnet objects to `series_id`.

BACnet devices → commission (poll loop) → `samples.csv` → bridge (ingest) → `feather_store`

Details: [BACnet polling](#), [Containers](#).

28.2 Rule evaluation

1. Rules are Python files in `workspace/data/rules_py/` with metadata in `rules_store.json`.
2. **Bindings** map rules to points via the Brick model (`fdd_input` tags).
3. Batch runner (`POST /api/rules/batch` or host timer) produces `fdd_results.json`.
4. Dashboard **faults** view and check-engine status read aggregated results.

28.3 Model sync

- Commissioning imports `update_model.json` / TTL graph.
- `POST /api/model/sync-ttl` refreshes SPARQL-backed tree used by the UI.

28.4 Retention

Feather files grow on disk; retention is configurable (workspace/data.env.local). Image upgrades must **preserve** workspace/data/ — backup before docker compose up -d --force-recreate. See [Updating the stack](#).

29 Python recipes (Arrow)

30 Python recipes (Arrow)

All Rule Lab Python rules use `apply_faults_arrow(table, cfg, context)` on PyArrow tables with `pyarrow.compute`.

See [Arrow recipes](#) for the canonical cookbook (flatline, spread, OOB, schedule faults).

Shared helpers live in `open_fdd.arrow_runtime.cookbook` and `open_fdd.arrow_runtime.windows`.

```
from open_fdd.arrow_runtime.cookbook import flatline_1h_mask

def apply_faults_arrow(table, cfg, context=None):
    return flatline_1h_mask(table, cfg)
```

31 Containers

32 Containers

Open-FDD v3 edge sites run **three** GHCR containers plus optional **host** services (Caddy, Ollama). There is no separate BACnet poll container — polling is part of **commission**.

32.1 GHCR images

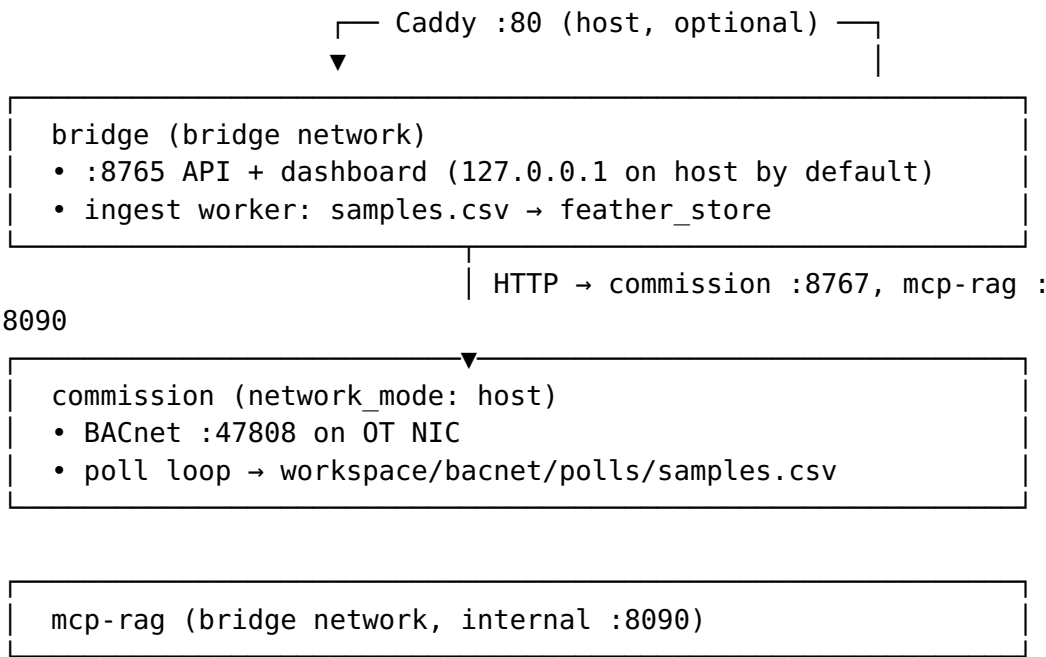
Image	Compose service	Role
ghcr.io/ bbartling/	bridge	Operator API, React dashboard,

Image	Compose service	Role
openfdd-bridge		feather ingest
ghcr.io/bbartling/openfdd-commission	commission	BACnet discover / read / write + poll loop
ghcr.io/bbartling/openfdd-mcp-rag	mcp-rag	Doc-search sidecar for agent tools

Tags: [GitHub Packages](#) (e.g. 2026.06.07-edge).

Template: docker/compose.edge.yml → copy to ~/open-fdd/docker-compose.yml.

32.2 Stack diagram



32.3 Responsibilities

Concern	Container
BACnet Who-Is / read / write	commission
BACnet scheduled RPM reads	commission (bacnet_poll_loop)
CSV → feather historian	bridge (bacnet_poll_worker thread)

Concern	Container
Operator login, Rule Lab, faults UI	bridge
Agent doc search	mcp-rag

The bridge thread named `bacnet_poll_worker` is **ingest only** — it does not bind BACnet.

32.4 Network and ports (typical edge)

Endpoint	Reachable from	Notes
Caddy :80	Building LAN	Reverse proxy to bridge
Bridge :8765	Loopback (default)	Bind 127.0.0.1 in compose
Commission : 8767	Bridge / loopback	HTTP API for BACnet ops
Commission : 47808	OT BACnet LAN	UDP; requires <code>network_mode: host</code>
MCP :8090	Bridge container network	Not exposed to LAN by default

32.5 Long-lived deployment

All compose services set `restart: unless-stopped`. After host reboot:

```
sudo systemctl enable docker
cd ~/open-fdd && docker compose ps
```

If containers are down: `docker compose up -d` (idempotent).

See [Run with Docker images](#).

32.6 Persistent data

State lives on the **host** under `workspace/` (bind mount). Containers are replaceable; **backup workspace/** before image upgrades.

Path	Contents
workspace/data/ feather_store/	Historian
workspace/data/*.json	Model, rules, FDD results
workspace/bacnet/ commissioning/	commission.env, points.csv
workspace/bacnet/ polls/samples.csv	Poll output
workspace/ auth.env.local	Login secrets
workspace/api/static/ app/	Dashboard bundle (if updated separately)

32.7 Optional host services

Service	Role
Caddy	TLS or plain HTTP :80 → 127.0.0.1:8765
Ollama	Optional LLM on :11434 (not in the three-image stack)
FDD loop timer	Optional openfdd-fdd- loop.timer on host (docker exec batch)

33 Architecture

34 Architecture

High-level view of the Open-FDD v3 edge stack: **three Docker containers** (bridge, commission, mcp-rag), BACnet polling inside **commission**, historian ingest in **bridge**.

Page	Topic
System overview	Components, diagram, deployment pointer
Data flow	Poll → CSV → feather → FDD → dashboard
Containers	

Page	Topic
	Images, networks, ports, persistence, reboot

Operators: deploy and upgrade via [Quick Start](#) — no git clone on the edge host.

BACnet polling: [Polling](#) — commission loop only (no separate poll container).

35 Rule Cookbook

36 Rule Cookbook

Practical patterns for **Arrow-native Rule Lab** ([apply_faults_arrow](#)) on feather historian PyArrow tables.

Page	Use when
Arrow recipes	Default — thresholds, flatline, spread, OOB, fan/schedule faults
Python recipes	Same Arrow patterns with shared <code>open_fdd.arrow_runtime.cookbook</code> imports
Windowing & debugging	Rolling windows, batch runtime, Rule Lab console

All expression rules are **PyArrow columnar** — no YAML rule files, no pandas DataFrames in Rule Lab.

Future graph ML (sklearn / PyG) runs in a separate training service — see [GitHub issue #211](#).

```
import pyarrow.compute as pc

def apply_faults_arrow(table, cfg, context=None):
    return pc.greater(table["zone_temp"], cfg["max_zone_temp"])
```

37 Windowing & debugging

38 Windowing & debugging

38.1 Rolling windows (Arrow)

Use `open_fdd.arrow_runtime.windows` for sample-based rolling min/max and consecutive-true streaks:

```
from open_fdd.arrow_runtime.windows import arrow_rolling_min,
arrow_rolling_max, arrow_consecutive_true
```

Cookbook masks (`flatline_1h_mask`, `spread_1h_mask`, `oob_mask`) default to ~12 samples (~1 h at 5 min poll).

Config keys: `flatline_window_samples`, `flatline_tolerance`, `max_spread`, `rolling_avg_minutes`, `bounds_low` / `bounds_high`.

38.2 Rule Lab console

Quick-test and batch responses include backend: `arrow`, `ms`, and flagged row counts. Arrow summary events appear in the event console on fault hits.

39 Dashboard overview

40 Dashboard overview

40.1 Primary areas

Area	Purpose
Home / Building insight	Check-engine status (green/yellow/red), active fault summary
Trends	Point history from feather store
Faults	Rule hits grouped by equipment family
Rule Lab	Edit, lint, and test Python rules

Area	Purpose
Data model	Equipment tree, point bindings, import health
BACnet	Discovery, reads, commission workflows
Settings / health	Stack status, auth mode

40.2 Live updates

Dashboard tiles use REST polling and WS `/ws/dashboard` for selected live views (ticket auth via POST `/api/auth/ws-ticket`).

40.3 Roles

Read-only operators see trends and faults; integrators access Rule Lab and model import. See [Auth and roles](#).

41 Auth and roles

42 Auth and roles

42.1 Production (LAN / edge)

- Set `OFDD_AUTH_SECRET` and role passwords in `workspace/auth.env.local`.
- Bridge on non-loopback addresses **requires** auth unless explicit insecure dev flags are set (lab only).

Role	Typical access
viewer	Read trends, faults, model tree
operator	Run batch FDD, acknowledge views
integrator	Rule Lab, bindings, model import
commission	BACnet writes (plus write guard)
agent	Agent tools / app-edit gates
admin	Full configuration

Login: POST `/api/auth/login` → Bearer JWT on API calls.

42.2 Localhost dev

OFDD_BRIDGE_HOST=127.0.0.1 with OFDD_AUTH_DISABLED=1 is acceptable on a single machine. **Never** use auth-disabled mode on a building LAN.

Full hardening: [Security](#).

43 Rule Lab

44 Rule Lab

Rule Lab authors **Arrow-native** Python rules against the feather historian. Rules define:

```
import pyarrow.compute as pc

def apply_faults_arrow(table, cfg, context=None):
    return pc.greater(table["zone_temp"], cfg["max_zone_temp"])
```

The bridge detects `apply_faults_arrow`, runs rules on **PyArrow tables** via `open_fdd.arrow_runtime`, and persists .py sources under `workspace/data/rules_py/`.

- **Quick test** — POST `/api/playground/test-rule` (returns backend: arrow, ms, flagged count)
- **Batch** — POST `/api/rules/batch` / `openfdd-fdd-loop` timer
- **Templates** — GET `/api/playground/arrow-templates`

Pin points to rules via **Model & assignments** (/model) commissioning JSON or BACnet tree right-click.

45 Model workflow

46 Model workflow

46.1 Brick model

- **Equipment tree** — SPARQL over synced TTL (`workspace/data/data_model.ttl`)

- **Point registry** — BACnet inventory → series_id for historian bindings
- **Health score** — orphan points, missing fdd_input, stale telemetry

46.2 Typical integrator flow

1. Import or sync BACnet point registry.
2. Map equipment classes (AHU, VAV, ...) in the model UI.
3. Tag points with fdd_input roles (zone temp, SAT, damper cmd, ...).
4. Bind Rule Lab rules to those inputs.
5. Run `POST /api/model/sync-ttl` after bulk imports.

Export: `GET /api/model/export` (TTL/JSON). API detail in [Appendix](#).

47 Operator Bridge

48 Operator Bridge

The Operator Bridge is the FastAPI service and React dashboard operators use daily.

Page	Topic
Dashboard overview	Main UI areas
Auth and roles	Login and permissions
Rule Lab	Python rule authoring
Model workflow	Brick bindings and imports

REST details: [Appendix — API routes](#).

49 Network setup

50 Network setup

50.1 Bind address

Commission and poll containers need the **OT interface** IP and UDP **47808**:

```
# workspace/bacnet/commissioning/commission.env
BACNET_BIND=192.168.1.50/24:47808
BACNET_INSTANCE=599999
```

- Use the NIC that can reach controllers (not only the management VLAN unless routed).
- Poll driver uses network_mode: host on Linux edges.

50.2 Discovery range

```
DISCOVER_LOW=1
DISCOVER_HIGH=4194302
DISCOVER_TIMEOUT=30
```

Narrow the range in large sites to avoid broadcast storms.

50.3 Verify

From commission container logs or API: Who-Is → I-Am responses. If zero devices, check firewall, bind IP, and VLAN routing before tuning rules.

51 Discover and read

52 Discover and read

The **commission** container exposes HTTP APIs for discovery and read-property workflows used by the dashboard BACnet tools.

Capability	Notes
Who-Is / I-Am	Device discovery on OT subnet
Read property	Present value, object name, units
RPM	Bulk reads for inventory export
Write property	Gated — see Write safety

Exported inventory feeds `points_discovered.csv` and the Brick point registry.

52.1 Operator workflow

1. Confirm [network setup](#).
2. Run discover from dashboard or API.
3. Review discovered objects; enable points for poll profiles.
4. Sync model bindings for FDD inputs.

53 Polling

54 Polling

BACnet polling has two stages:

1. **Field reads** — commission poll loop → `workspace/bacnet/polls/samples.csv`
2. **Historian ingest** — bridge background worker → `workspace/data/feather_store/`

54.1 Commission poll loop (default)

The **commission** container runs the poll loop at startup. It reads enabled rows from `points.csv` and appends BACnet RPM results to `samples.csv`.

Setting	Location
BACnet bind	<code>workspace/bacnet/commissioning/commission.env</code>
Enabled points	<code>workspace/bacnet/commissioning/points.csv</code>
Poll output	<code>workspace/bacnet/polls/samples.csv</code>
Historian	<code>workspace/data/feather_store/</code>

Check activity:

```
tail -1 workspace/bacnet/polls/samples.csv
docker compose logs --since 5m commission | grep -i poll
```

54.2 Bridge ingest worker

Inside **bridge**, a background thread watches `samples.csv` and ingests new rows into feather. Disable only for debugging:
`OFDD_DISABLE_POLL_WORKER=1`.

54.3 Health

Dashboard stack health (GET `/health/stack`) reports `bacnet_poll` status from the commission agent — not a separate container.

Stale points trigger data-quality fault patterns — see [Sensor quality faults](#).

55 Write safety

56 Write safety

BACnet writes can change live plant behavior. Open-FDD disables writes by default. Enable only with operator sign-off, a tested allowlist, and audit logging.

56.1 Defaults

Control	Default
Supervisory writes	Off
Write allowlist	Empty — must enumerate device/object
Audit	Commands logged when writes enabled

56.2 Enable writes (commission role)

1. Set env flag documented in [Security](#) (`OFDD_BACNET_WRITE_ENABLED` or site equivalent).
2. Populate write allowlist CSV / config with explicit object IDs.
3. Use **commission** role JWT; every write should include reason/ note where API supports it.

4. Test on bench hardware before production OT.

56.3 Operator rules

- Never write setpoints or overrides on life-safety or fire/smoke interlocks through experimental rules.
- Prefer read-only polling for FDD until rules are validated in Rule Lab.
- Revert overrides through BACnet priority relinquish where supported.

57 BACnet

58 BACnet

Open-FDD integrates with field controllers via BACnet/IP (and bench MSTP overlays in dev compose).

Page	Topic
Network setup	Bind address and NIC
Discover and read	Commission agent
Polling	Historian driver
Write safety	Defaults and allowlists

59 Convention

60 Fault code convention

60.1 Format

FAMILY-SUFFIX — suffix is **1-3 letters** (e.g. VAV-C, AHU-B). Numeric suffixes like VAV-03 are avoided (collision with physical equipment names).

60.2 Categories

Category	Meaning
performance_degradation	Efficiency or capacity drift
simultaneous_heat_cool	Heating and cooling fighting
sensor_fault	Flatline, OOB, inconsistent readings
io_fault	Command vs feedback mismatch

60.3 Severity

Level	Operator response
info	Log; trend for maintenance window
warning	Investigate within days
critical	Prompt attention; may affect comfort/energy significantly

60.4 Linking rules

In Rule Lab, set `fault_code` on the rule metadata. Batch FDD aggregates hits into GET `/api/faults/status` by family.

60.5 Cookbook mapping

Each catalog entry lists `cookbook_patterns` (e.g. `flatline_1h`) — see [Rule Cookbook](#).

61 AHU faults

62 AHU faults

Air handling unit codes (sample from catalog).

Code	Title	Severity	Detection summary	Likely causes	Operator checks
AHU-A	Supply fan performance degradation	warning	Spread/runtime vs baseline	Belt slip, dirty filters, VFD limit	Trend speed vs airflow; filter dP

Code	Title	Severity	Detection summary	Likely causes	Operator checks
AHU-B	Simultaneous heating and cooling	critical	Heat + cool outputs active together	Sequencing bug, leaking valve	Trend coil commands; valve feedback
AHU-C	SAT sensor fault	warning	Flatline / OOB SAT	Failed sensor, bad calibration	SAT trend; compare to coil discharge
AHU-D	MAT inconsistent with OAT/RAT	warning	MAT outside OAT-RAT envelope	MAT/OAT/RAT sensor errors	Verify damper positions; cross-check OAT
AHU-E	Economizer not economizing	warning	Free cool available, OA minimum	Stuck damper, lockout	OA damper vs OAT trend
AHU-F	Damper command vs feedback mismatch	warning	Cmd $\neq$ feedback	Stuck actuator, linkage	Compare cmd/feedback trends

Example rule: flatline_1h on SAT $\rightarrow$ tag **AHU-C**. Recipe: [Python recipes](#).

63 VAV faults

64 VAV faults

Variable air volume terminal codes (sample).

Code	Title	Severity	Detection summary	Likely causes	Operator checks
VAV-A	Zone heating/cooling overlap	warning	Heat and cool at terminal	Reheat valve stuck, SAT too low	Zone temp vs SAT; valve cmd
VAV-B	Poor zone temperature control	warning	Persistent deviation	Oversized box, bad tuning	PI loop trends; occupancy

Code	Title	Severity	Detection summary	Likely causes	Operator checks
			from setpoint		
VAV-C	Zone temperature sensor fault	warning	Flatline / OOB zone temp	Failed zone sensor	Flatline test; compare to neighbor zones
VAV-D	Minimum airflow not met	warning	Flow below minimum cmd	Damper stuck, pressure issue	Airflow vs damper cmd
VAV-E	Discharge temp deviation from SAT	warning	DAT far from SAT setpoint	Coil issue, sensor drift	DAT vs SAT SP

Example rule: oob_rolling on zone temp → **VAV-C** or comfort bounds per site policy.

65 RTU faults

66 RTU faults

Packaged rooftop unit codes (sample).

Code	Title	Severity	Detection summary	Likely causes	Operator checks
RTU-A	Compressor short cycling	warning	Rapid on/off cycles	Low charge, oversized unit	Run-time histogram
RTU-B	Economizer / OA damper fault	warning	OA behavior vs conditions	Actuator, sensor	OA damper vs enthalpy/OAT
RTU-C	Supply air temperature fault	warning	SAT flatline or OOB	Sensor failure	SAT trend vs outdoor conditions

Code	Title	Severity	Detection summary	Likely causes	Operator checks
RTU-D	Simultaneous heat and cool	critical	Heat + cool stages together	Control board, relay stuck	Stage status trends

Bind rules to packaged unit fdd_input points (SAT, OAT, compressor cmd, ...).

67 Sensor and data quality

68 Sensor and data quality

Code	Title	Severity	Detection summary	Likely causes	Operator checks
BLD-B	Outdoor air sensor fault	warning	OAT flatline / unrealistic	Sensor exposure, failed probe	Compare to weather service
BLD-D	Stale telemetry	warning	No new samples within threshold	Poll driver down, device offline	Poll health; BACnet comms
DC-C	Datacom sensor anomaly	warning	CRAH/room sensor flatline	Failed sensor	Row-level flatline recipe

68.1 Communication quality

- Monitor poll cycle timestamps in dashboard.
- Use stale_points cookbook pattern for dropout detection.
- Do not confuse **comm loss** with **equipment fault** — verify BACnet device online before dispatching mechanical.

68.2 Alarm console quality

When integrating with BMS alarms (future/external), map Open-FDD codes to operator work orders — Open-FDD itself is **analytics-first**, not a dial-out alarm server.

69 Fault Codes

70 Fault Codes

Open-FDD uses a **check-engine** model: one building-level status (green / yellow / red) with a catalog of fixed fault codes per equipment family.

Page	Content
Convention	Naming, severity, categories
AHU faults	Air handling units
VAV faults	Terminal units
RTU faults	Packaged rooftops
Sensor & data quality	Stale, flatline, comms

Live catalog API: GET /api/faults/catalog on the Operator Bridge.

71 Live site update (SSH)

72 Live site update (SSH)

IT operators: prefer [Quick Start — Updating the stack](#) and the helper scripts `openfdd_site_backup.sh` / `openfdd_site_update.sh`. This page is the detailed SSH runbook with manual commands.

Use this runbook when a **live edge VM** already has a minimal deploy folder and you are **not** doing `git pull` on the host.

Typical layout (no full git checkout):

```
/home/bbartling/open-fdd/  
  docker/
```

```
docker-compose.yml
workspace/
```

workspace/ is the **live site state** — BACnet commissioning files, poll CSV, feather historian, BRICK model, FDD rules, MCP RAG index, and env files. Image upgrades must **preserve** it.

72.1 Concepts (plain language)

Term	Meaning
Container	A running app instance (bridge, commission, mcp-rag). Recreated on upgrade.
Image	Downloaded app package from GHCR (e.g. openfdd-bridge:2026.06.07-edge).
Docker volume	Persistent data Docker manages separately. This minimal layout uses bind mounts into workspace/ instead.
workspace/	Open-FDD site state on disk. This is what you must not lose.

Never on live sites: `docker compose down -v`, `docker volume prune`, `docker system prune -a --volumes`, or `docker rm -f $` (`docker ps -aq`). Those can destroy data or leave the site down with no rollback path.

72.2 Before you start

1. Maintenance window or low-traffic period (containers restart briefly).
2. SSH access to the edge host.
3. New GHCR tag published (check [GitHub Packages](#)).
4. Optional: control machine with the full open-fdd repo for post-deploy check.

72.3 1. Backup workspace/

Container processes may write files as **root:root** with mode **0600**. A normal user tar can fail on those paths — use sudo:

```
cd /home/bbartling/open-fdd

export BACKUP_ROOT="$HOME/openfdd-backups/$(date +%Y%m%d-%H%M%S)"
mkdir -p "$BACKUP_ROOT"

cp docker-compose.yml "$BACKUP_ROOT/docker-compose.yml.before"
```

```
docker compose ps > "$BACKUP_ROOT/docker-compose-ps-before.txt"
docker compose config --images > "$BACKUP_ROOT/docker-images-
before.txt"
```

```
sudo tar --xattrs --acls -czf "$BACKUP_ROOT/workspace-full.tgz"
workspace
sudo chown "$USER:$USER" "$BACKUP_ROOT/workspace-full.tgz"

echo "Backup saved to: $BACKUP_ROOT"
du -h "$BACKUP_ROOT/workspace-full.tgz"
```

72.4 2. Verify new images exist on GHCR

```
export NEW_TAG="2026.06.07-edge"

docker manifest inspect ghcr.io/bbartling/openfdd-bridge:${
{NEW_TAG}} >/dev/null &&
docker manifest inspect ghcr.io/bbartling/openfdd-commission:${
{NEW_TAG}} >/dev/null &&
docker manifest inspect ghcr.io/bbartling/openfdd-mcp-rag:${
{NEW_TAG}} >/dev/null &&
echo "All images exist for ${NEW_TAG}"
```

Replace NEW_TAG with the tag you intend to deploy.

72.5 3. Update the React UI bundle (required for dashboard changes)

Image pull alone does not update the browser UI. The bridge serves workspace/api/static/app/ from the **bind mount first**, before anything baked into the GHCR image (static_dashboard_dir() in the bridge). If that folder still has an old index-*.js hash (e.g. index-TRH4YIfA.js), you will keep seeing the old BACnet tree, Host Stats revisions panel, Model & assignments tab, etc.

From bensserver (control machine, full repo):

```
cd /path/to/open-fdd
./scripts/build_operator_dashboard.sh prod
cd infra/ansible
./deploy.sh ui --limit acme_vm_bbartling
```

Confirm the edge picked up the new hash:

```
curl -sf http://<edge-ip>/ | grep -o 'index-[^\"]*\.\js' | head -1
# Should match bensserver: grep -o 'index-[^\"]*\.\js' workspace/
api/static/app/index.html
```

One command (recommended after CI publishes a new tag):

```
OPENFDD_IMAGE_TAG=2026.06.08-edge ./scripts/upgrade_edge_full.sh
--limit acme_vm_bbartling
```

That runs: dashboard build → deploy.sh ui → GHCR pull + docker
compose up -d --force-recreate → post_deploy_check.sh --full.

72.6 4. Update docker-compose.yml and recreate containers

Edit image tags in docker-compose.yml (example: 2026.06.04-edge → 2026.06.07-edge):

```
cd /home/bbartling/open-fdd

cp docker-compose.yml "$BACKUP_ROOT/docker-compose.yml.before-tag-
change"
sed -i "s/2026\.06\.04-edge/${NEW_TAG}/g" docker-compose.yml

docker compose config --images
docker compose pull
docker compose up -d --force-recreate

docker compose ps
```

Bind-mounted workspace/ is unchanged; only container images restart.

72.7 5. On-VM smoke checks

```
cd ~/open-fdd

echo "=== Stack ==="
docker compose ps
docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}"

echo "=== Bridge health ==="
curl -sf http://127.0.0.1:8765/health | jq . || curl -sf http://
127.0.0.1:8765/health

echo "=== Stack health (if available – may require auth) ==="
curl -i http://127.0.0.1:8765/health/stack

echo "=== MCP health from inside mcp-rag container ==="
```

```

    docker compose exec mcp-rag sh -lc 'curl -sf http://
127.0.0.1:8090/health || wget -qO- http://127.0.0.1:8090/health ||
python - <<PY
    import urllib.request
    print(urllib.request.urlopen("http://127.0.0.1:8090/health",
timeout=5).read().decode())
PY'

echo "=== BACnet samples still moving ==="
ls -lah workspace/bacnet/polls/samples.csv
tail -n 1 workspace/bacnet/polls/samples.csv

echo "=== BACnet ingest state ==="
sudo cat workspace/data/bacnet_ingest_state.json | jq . || sudo
cat workspace/data/bacnet_ingest_state.json

echo "=== Feather store retained and writing ==="
du -sh workspace/data/feather_store
find workspace/data/feather_store -type f -printf "%TY-%Tm-%Td
%TH:%TM %p\n" | sort | tail -n 10

echo "=== Recent log errors ==="
docker compose logs --since 20m | grep -Ei "error|exception|
traceback|critical|failed|permission|denied" || true

```

/health/stack nuance: /health should return 200 without auth. /health/stack may return 401, 403, 404, or an empty body depending on auth settings — use curl -i (not curl -sf) so you see the status code. A failed silent curl -sf does not mean the stack is broken.

72.7.1 Arrow / FDD rules (Open-FDD 3.0+)

All saved rules use apply_faults_arrow(table, cfg, context) on PyArrow tables.

From bensserver:

```
./scripts/validate_fdd_backends.sh --docker # must exit 0
```

On the edge VM after upgrade:

```

grep -R "apply_faults_arrow" -n workspace/data/rules_py 2>/dev/
null | wc -l
docker compose exec bridge python3 -c "
from openfdd_bridge.rule_store import RuleStore
from open_fdd.arrow_runtime.rules import detect_rule_backend
from openfdd_bridge.rule_source import read_source
for r in RuleStore().list_rules():
    if not r.get('enabled', True): continue
    code = read_source(r.get('source_path', '')) or
r.get('code', '')

```

```
        print(r.get('name'), detect_rule_backend(code, r))
    "
```

Spot-check **Rule Lab** quick-test shows backend: arrow.

72.8 6. Control-machine insurance check

From your laptop or bensserver (full repo clone, not on the edge VM):

```
cd /path/to/open-fdd/infra/ansible

./scripts/post_deploy_check.sh --limit acme_vm_bbartling

# Or explicitly:
./scripts/post_deploy_check.sh --host 100.122.106.124 --ssh-user
bbartling
```

This is the recommended **non-destructive** check: Caddy → React dashboard, /health/stack, MCP, BACnet tree, BRICK/SPARQL, Docker log scan.

Do not run on live commissioned sites:

openfdd_edge_validate.sh --full or acme_operational_verify.sh
— those rediscover BACnet and reset bench model/rules.

72.9 7. Browser validation

Open the site via **Caddy on port 80** (e.g. http://<tailscale-or-lan-ip>/), not only 127.0.0.1:8765:

1. View page source / Network — confirm **new** index-*.js hash (not the pre-upgrade bundle).
2. Home / faults dashboard loads.
3. Integrator login (workspace/auth.env.local).
4. **Host stats** — container image tag / git sha (3.0+ bridge).
5. **BACnet** — right-click point → **Refresh PV, Read priority array**.
6. **Model & assignments** (/model) — commissioning JSON export (404 = bridge image + UI both stale).
7. Rule Lab — quick-test a saved rule (backend: arrow).

72.10 Rollback

If the new tag misbehaves, restore the previous tag in compose and recreate:

```

cd /home/bbartling/open-fdd

# Example: roll back from 2026.06.07-edge to 2026.06.04-edge
export NEW_TAG="2026.06.07-edge"
export OLD_TAG="2026.06.04-edge"

sed -i "s/${NEW_TAG}/${OLD_TAG}/g" docker-compose.yml

docker compose pull
docker compose up -d --force-recreate

docker compose ps
curl -sf http://127.0.0.1:8765/health | jq . || curl -sf http://
127.0.0.1:8765/health

```

workspace/ is unchanged by rollback. For data corruption, extract files from \$BACKUP_ROOT/workspace-full.tgz selectively (model, rules, feather shards) — do not blindly overwrite a running site without stopping containers first.

72.11 Alternative: Ansible image-only upgrade

If you have inventory and SSH from a control machine (no manual sed on the VM):

```

export OPENFDD_IMAGE_TAG=2026.06.07-edge
RUN_POST_CHECK=1 ./scripts/upgrade_edge_ghcr.sh --limit
acme_vm_bbartling

```

See also [Updating the stack](#).

73 LAN hardening

74 LAN hardening

74.1 Deployment modes

Mode	Bind	Auth
Local dev	127.0.0.1	Optional 0FDD_AUTH_DISABLED=1
Lab LAN (insecure)	0.0.0.0	

Mode	Bind	Auth
		Requires explicit insecure dev flags — not production
Edge production	0.0.0.0 behind Caddy	Required credentials + OFDD_AUTH_SECRET

Startup **fails** on non-loopback bind without credentials (unless insecure lab flags are explicitly set).

74.2 Network exposure

- Do not port-forward the bridge to the public internet without TLS and strong auth.
- Keep BACnet on OT VLANs; bridge management on IT VLAN with firewall rules.
- Prefer Caddy on :80/443 rather than exposing :8765 directly.

74.3 Rule Lab caution

Rule Lab executes **operator-authored Python** in a sandbox. Restrict integrator accounts; review rules before enable on production sites.

75 Secrets and auth

76 Secrets and auth

76.1 Files (gitignored)

File	Contents
workspace/ auth.env.local	JWT secret, role passwords
infra/ansible/secrets/ <host>.env.local	SSH deploy secrets (maintainers)

Copy from *.example templates only.

76.2 Required variables (production)

Variable	Purpose
OFDD_AUTH_SECRET	HMAC signing (32+ chars)
OFDD_INTEGRATOR_USER / PASSWORD	Rule Lab, bindings
OFDD_OPERATOR_USER / PASSWORD	Operations

76.3 GHCR pulls

Use read-only registry credentials on edge if images are private; rotate tokens periodically.

76.4 Backup

Back up workspace/data/ and auth env in secure operator vault — not in git.

77 BACnet write allowlists

78 BACnet write allowlists

Supervisory writes require:

1. **Feature flag** enabled in environment (off by default).
2. **Allowlist** file at workspace/bacnet/write_allowlist.json (copy from `write_allowlist.example.json`). If the flag is on but the allowlist is missing, writes are **denied**.
3. **Integrator** role on API calls.
4. **Audit** log review after commissioning sessions.

Allowlist entries support `device_instance`, `object_identifier`, `property_identifier`, optional `priority_min/priority_max`, `value_min/value_max`, and `allowed_values` for binary/multistate targets.

Lab-only override: `OFDD_BACNET_WRITE_ALLOW_ANY=1` (audited; not for production edges).

See operator guide: [BACnet write safety](#).

Never enable blanket write access for agent or integrator automation without human approval per site.

79 Operations

80 Operations

Runbooks for live edge hosts after the initial deploy.

Page	When to use
Live site update (SSH)	Minimal ~/open-fdd/ folder on a VM — pull new GHCR tags, preserve workspace/

For first-time deploy from a control machine, see [Quick Start — Docker](#).

81 Security

82 Security

Concise deployment security for the Operator Bridge and BACnet tools.

Page	Topic
LAN hardening	Auth and bind modes
Secrets and auth	Env files
BACnet write allowlists	Supervisory write gates

83 API routes

84 API routes

REST API served by **openfdd-bridge** (default <http://127.0.0.1:8765>).
Production: Caddy on :80 proxies to the bridge.

OpenAPI: GET /docs and GET /redoc when the bridge runs.

Auth: JWT via POST /api/auth/login. Most routes require Authorization: Bearer <token>. Roles: viewer, operator, integrator, commission, agent, admin. BACnet writes require commission role + write safety. Dev: OFDD_AUTH_DISABLED on loopback only.

WebSocket: WS /ws/dashboard — POST /api/auth/ws-ticket, then pass the ticket via Sec-WebSocket-Protocol: ofdd.ws, <ticket>. Query ?ticket= is dev-only when OFDD_WS_ALLOW_QUERY_TICKET=1. Tickets are short-lived and not interchangeable with the Bearer token.

84.1 Health & audit

Method	Path	Role	Description
GET	/health	public	Minimal liveness (ok, service, version, auth_required)
GET	/health/stack	auth	Stack traffic-light (verbose URLs/bind with OFDD_DEBUG_DIAGNOSTICS)
POST	/api/auth/ws-ticket	auth	Short-lived WebSocket ticket
GET	/api/audit/summary	auth	Audit counters
GET	/api/audit/events	auth	Audit log (query params)
GET	/api/audit/errors	auth	Recent API errors

*Exact public set depends on OFDD_AUTH_DISABLED and middleware.

84.2 Auth

Method	Path	Description
POST	/api/auth/login	Username/password → JWT
GET	/api/auth/me	Current user
GET	/api/auth/status	Auth mode / logout hints

84.3 Rules & FDD

Method	Path	Role	Description
GET	/api/rules/saved	user	Rule index
POST	/api/rules/save	operator+	Create/update rule metadata
GET	/api/rules/saved/{id}/source	user	Python source
PUT	/api/rules/saved/{id}/source	operator+	Write .py file
GET	/api/rules/assignments	user	Point bindings
POST	/api/rules/bind	integrator+	Bind rule to point
POST	/api/rules/bindings	integrator+	Bulk bindings
POST	/api/rules/batch	operator+	Run all enabled rules → fdd_results.json
GET/POST	/api/rules/drafts	user	Draft rules

84.4 Rule Lab playground

Method	Path	Description
POST	/api/playground/lint	AST lint Python rule
POST	/api/playground/test-rule	Run apply_faults_arrow() on feather PyArrow window
POST	/api/playground/run-script	Execute script mode
GET	/api/playground/sample-frame	Sample DataFrame for editor

84.5 BRICK / data model

Method	Path	Description
GET	/api/model/sites	Site list
GET	/api/model/tree	Equipment tree (SPARQL)
GET	/api/model/graph	RDF graph fragment
GET	/api/model/scope	Scoped query
GET	/api/model/export	Export TTL/JSON
GET	/api/model/health	Model point/equipment counts
GET	/api/model/ttl	TTL metadata
POST	/api/model/sync-ttl	Regenerate TTL from JSON
POST	/api/model/import	Import model payload
POST	/api/model/sites	Create site
GET/ POST	/api/model/bacnet-sync	BACnet ↔ model sync state

84.6 BACnet

Method	Path	Role	Description
GET	/config/bacnet	read	Commission URL / config
GET	/api/bacnet/commission/status	read	Commission agent health
GET	/api/bacnet/server/points	read	Server point table
GET	/api/bacnet/inventory	read	Discovered inventory
POST	/api/bacnet/whois	commission	Who-Is
POST	/api/bacnet/discover	commission	Discover devices
POST	/api/bacnet/read	commission	Read property
POST		commission	RPM read

Method	Path	Role	Description
	/api/ bacnet/ read- multiple		
POST	/api/ bacnet/ write	write	Supervisory write (gated)
POST	/api/ bacnet/ import-to- model	integrator	Inventory → BRICK
POST	/api/ bacnet/ driver/ sync- discovery	commission	Sync driver tree
POST	/api/ bacnet/ poll/once	commission	Trigger poll cycle
GET	/api/ bacnet/ poll/status	read	Poll worker status
POST	/ingest/ bacnet	read	Ingest samples into feather

Bind address: BACNET_BIND in commission env — see [BACnet network setup](#). Operator guide: [Discover and read](#).

PATCH | /api/bacnet/driver/point | commission | Enable poll + interval on point |
 PATCH | /api/bacnet/driver/device | commission | Enable poll for all points on device |
 PATCH | /api/bacnet/driver/device/remap | commission | Change instance and/or address |
 DELETE | /api/bacnet/driver/point/{point_id} | commission | Remove point from registry |
 DELETE | /api/bacnet/driver/device/{device_instance} | commission | Remove device |
 DELETE | /api/bacnet/driver/registry | commission | Clear CSVs + model sync |

84.7 Faults (check-engine)

Method	Path	Description
GET	/api/faults/catalog	Fixed fault codes

Method	Path	Description
GET	/api/faults/status	GREEN/YELLOW/RED aggregate
GET	/api/faults/tree	Fault tree by equipment
GET	/api/faults/code/{code}	Code metadata

84.8 Timeseries & sites

Method	Path	Description
GET	/api/timeseries/sites	Feather sites
GET	/api/timeseries/series	Series metadata
GET	/api/timeseries/plot	Plot payload
GET	/api/timeseries/readings	Raw readings
GET	/api/sites	Site list
GET	/api/sites/{site_id}/frame	Wide frame for site

84.9 Building & host

Method	Path	Description
GET	/api/building/status	Building summary
GET	/api/building/alerts	Active alerts
GET	/api/host/stats	CPU/mem/disk

84.10 Local agent (Ollama)

Prefix **/openfdd-agent**. Role **agent** for tool execution.

Method	Path	Description
GET	/openfdd-agent/context	Composed prompt context
GET	/openfdd-agent/tools	Tool manifest
POST	/openfdd-agent/tool	Execute tool (e.g. rules.save)
GET	/openfdd-agent/	Ollama reachability

Method	Path	Description
	ollama/ health	
GET	/openfdd-agent/ building-insight	Check-engine narrative context
GET	/openfdd-agent/zone-temps	Zone temperature insight
POST	/openfdd-agent/chat	Chat completion (catalog-bound)

Optional host Ollama for building insight — not required for core FDD. Configure via compose profile or host service; see [Architecture overview](#).

84.11 Modbus

Method	Path	Description
POST	/api/modbus/read_registers	Read holding registers

84.12 PyPI package (separate)

Library-only HTTP is **not** bundled. See [Python package](#) for `open_fdd.arrow_runtime` (Arrow rules offline).

84.13 Errors

JSON error bodies; stack traces hidden unless `OFDD_DEBUG_TRACEBACKS=1`. See [LAN hardening](#).

85 Python package

86 Python package (open-fdd)

Install from PyPI:

```
pip install open-fdd
# Arrow runtime + playground (default 3.0.1+)
pip install "open-fdd[ml]"
# optional numpy/sklearn for offline graph ML experiments
```

86.1 Modules

Module	Purpose
open_fdd.arrow_runtime	Default — apply_faults_arrow on PyArrow Tables, cookbook masks, column maps
open_fdd.playground	Rule Lab lint/compile helpers for Arrow rules

Retired in 3.0.1+: open_fdd.engine (YAML/pandas RuleRunner) is **not** shipped on PyPI. Use Operator Bridge Rule Lab or import rules from workspace/data/rules_py/.

86.2 When to use package-only

Scenario	Use
Lint/test Arrow rules offline	open_fdd.arrow_runtime + open_fdd.playground
Graph ML experiments (Layer B)	open-fdd[ml] in experiments/ (see issue #211)
Full building operator stack	Docker Operator Bridge

86.3 Versioning

Publish: git tag open-fdd-vX.Y.Z → GitHub Actions **Publish open-fdd**.

```
import open_fdd
print(open_fdd.__version__)
```

86.4 Offline Arrow example

```
import pyarrow as pa
import pyarrow.compute as pc
from open_fdd.arrow_runtime import run_arrow_rule

code = '''
import pyarrow.compute as pc

def apply_faults_arrow(table, cfg, context=None):
    return pc.greater(table["SAT"], float(cfg["high"]))
'''

table = pa.table({"SAT": [70.0, 90.0, 88.0]})
result = run_arrow_rule(code, table, {"high": 85})
print(result.flagged_count)
```

API surface: `open_fdd/arrow_runtime` docstrings and [Arrow recipes](#).

87 Configuration reference

88 Configuration reference

88.1 Bridge / auth

Variable	Default	Notes
OFDD_BRIDGE_HOST	127.0.0.1	0.0.0.0 for LAN
OFDD_AUTH_SECRET	—	Required for LAN
OFDD_AUTH_DISABLED	0	Localhost dev only
OFDD_INSECURE_LAN_DEV	0	Lab only with auth disabled

88.2 BACnet

Variable	Notes
BACNET_BIND	ip/mask:47808 in commission.env
OFDD_BACNET_WRITE_ENABLED	

Variable	Notes
	Default off — site-specific name may vary

88.3 Docker edge

Variable	Notes
OPENFDD_IMAGE_TAG	GHCR tag for Ansible deploy
openfdd_docker_pull_from_ghcr	Ansible host_var

Rule Lab Python rules use `rules_store.json + workspace/data/rules_py/*.py`. Legacy YAML schema (`docs/config_schema.json`) is archived — not used by Operator Bridge 3.0.1+.

89 Glossary

90 Glossary

Term	Definition
Operator Bridge	FastAPI service serving API + static dashboard
Rule Lab	UI and APIs for Arrow <code>apply_faults_arrow()</code> rules
Feather store	On-disk columnar historian under <code>workspace/data/feather_store/</code>
fdd_input	Brick tag linking a point to a rule binding role
Check-engine	Building-level GREEN/YELLOW/RED fault summary
Commission	BACnet discovery/read (and optional write) agent container
Poll driver	Scheduled BACnet RPM reads into historian
GHCR	GitHub Container Registry — <code>ghcr.io/bbartling/openfdd-*</code>

91 Appendix

92 Appendix

Reference material for integrators and library users.

Page	Content
API routes	Operator Bridge REST/WebSocket
Python package	PyPI open-fdd
Configuration reference	Key env variables
Glossary	Terms

93 Edge Docker deploy

94 Edge Docker deploy

Open-FDD edge hosts run **three** GHCR containers (bridge, commission, mcp-rag) with workspace/ bind-mounted for site state.

Task	Doc
First deploy	Quick Start — Run with Docker images
Backup + image upgrade	Quick Start — Updating the stack
Container architecture	Architecture — Containers
BACnet polling	BACnet — Polling

Typical host layout:

```
~/open-fdd/  
  docker-compose.yml    # from docker/compose.edge.yml  
  workspace/            # historian, BACnet, auth – backup before  
upgrades
```

Compose template: docker/compose.edge.yml in the repo.

95 Arrow-native runtime

96 Arrow-native runtime

Open-FDD 3.0 executes Rule Lab rules on **PyArrow tables** end to end.

```
feather_store.read_site_table()
  → fdd_runner / playground.run_arrow_table()
  → apply_faults_arrow(table, cfg, context)
  → pyarrow.compute masks
```

96.1 Authoring contract

```
import pyarrow.compute as pc

def apply_faults_arrow(table, cfg, context=None):
    return pc.greater(table["zone_temp"], cfg["max_zone_temp"])
```

Cookbook helpers: `open_fdd.arrow_runtime.cookbook` (flatline, spread, OOB, after-hours fan).

Script-mode analytics rules receive table (PyArrow) and `cfg` in the sandbox — not pandas DataFrames.

96.2 Package layout

Module	Role
<code>open_fdd.arrow_runtime.backend</code>	Execute rule code, batch chunks
<code>open_fdd.arrow_runtime.cookbook</code>	Shared fault masks
<code>open_fdd.arrow_runtime.windows</code>	Rolling min/max, consecutive-true
<code>open_fdd.playground.arrow_templates</code>	Rule Lab starter templates